

Permutaciones semánticamente válidas de una secuencia de asignaciones puras

Un enfoque mediante dependencias, DAGs y ordenamientos topológicos

Este artículo presenta, de principio a fin, una solución exacta —bajo hipótesis estándar de pureza y ausencia de aliasing— al problema de enumerar todas las permutaciones de una secuencia de asignaciones que preservan la semántica del programa. Se definen formalmente las nociones de lectura y escritura, se introducen las dependencias RAW, WAR y WAW, se construye el grafo de precedencia, se demuestra que es acíclico y se prueba que las permutaciones válidas son exactamente sus ordenamientos topológicos.

Abril de 2026

Resumen

Dada una secuencia lineal de asignaciones de la forma $x := e$, donde los lados derechos son puros, deterministas y sin efectos laterales, estudiamos el problema de caracterizar todas las permutaciones que preservan la semántica del programa. Mostramos que el problema se reduce a extraer un conjunto de restricciones de precedencia a partir de los conjuntos de variables leídas y escritas por cada instrucción. Dichas restricciones inducen un grafo dirigido acíclico; las secuencias semánticamente válidas son exactamente los ordenamientos topológicos de ese grafo. El desarrollo es autocontenido: fija el modelo semántico, presenta los algoritmos, incluye pruebas de corrección y recorre ejemplos completos.

1 Introducción

Considérese una secuencia de asignaciones

$$s_1; s_2; \dots; s_n,$$

donde cada instrucción tiene la forma

$$s_i \equiv x_i := e_i.$$

La pregunta que motiva este trabajo es la siguiente:

¿Cómo generar todas las permutaciones de $(s_1, \dots, s_n)$ que preservan la semántica del programa?

La intuición es simple. Dos instrucciones no pueden intercambiarse libremente si una produce, consume o sobrescribe información relevante para la otra. Esa intuición se formaliza mediante tres clases de dependencias: *read-after-write* (RAW), *write-after-read* (WAR) y *write-after-write* (WAW). A partir de ellas se obtiene un orden parcial sobre las instrucciones, representado como un DAG. Las permutaciones válidas resultan ser exactamente las extensiones lineales de ese orden parcial.

El interés del problema va más allá de este caso puntual. El mismo patrón conceptual aparece en scheduling, compilación, optimización, análisis de dependencias y transformaciones semánticamente seguras.

2 Hipótesis del modelo

Trabajaremos bajo las siguientes hipótesis:

- (H1) Cada instrucción es una asignación simple $x := e$.
- (H2) El lado derecho e es puro, determinista y sin efectos laterales.
- (H3) No hay aliasing, punteros, arrays mutables, referencias compartidas, E/S, excepciones observables ni concurrencia.
- (H4) La semántica observable del programa es el estado final de las variables.

Bajo estas suposiciones, la interacción entre instrucciones queda completamente descrita por las variables que cada una *lee* y *escribe*. Si alguna de estas hipótesis se abandona, el análisis puede requerir nociones adicionales de dependencia, por ejemplo sobre memoria compartida o efectos observables.

3 Lenguaje y semántica

3.1 Variables, estados y expresiones

Sea Var un conjunto de variables y Val un dominio de valores. Un *estado* es una función

$$\sigma : Var \rightarrow Val$$

que asigna un valor a cada variable.

Dada una expresión e , denotaremos por

$$\llbracket e \rrbracket_\sigma$$

su evaluación en el estado σ . Como las expresiones son puras, evaluarlas no modifica el estado y su resultado depende únicamente de los valores de las variables libres que aparecen en e .

3.2 Semántica de una asignación

La ejecución de una asignación

$$x := e$$

sobre un estado σ produce el estado σ' definido por

$$\sigma'(x) = \llbracket e \rrbracket_\sigma, \quad \sigma'(y) = \sigma(y) \text{ para todo } y \neq x.$$

3.3 Semántica de una secuencia

Una secuencia

$$s_1; \dots; s_n$$

se interpreta por composición de transformaciones de estado. Dos secuencias serán *semánticamente equivalentes* si, para todo estado inicial σ , producen exactamente el mismo estado final.

4 Formulación del problema

Sea

$$S = (s_1, \dots, s_n)$$

una secuencia fija de instrucciones. Una *permutación* de S es una reordenación

$$S' = (s_{\pi(1)}, \dots, s_{\pi(n)})$$

donde π es una permutación de $\{1, \dots, n\}$.

Definición 4.1 (Permutación semánticamente válida). *Diremos que S' preserva la semántica de S si, para todo estado inicial σ , la ejecución de S y la ejecución de S' producen el mismo estado final.*

Nuestro objetivo es caracterizar y enumerar *todas* las permutaciones semánticamente válidas.

5 Conjuntos de lectura y escritura

A cada instrucción

$$s_i \equiv x_i := e_i$$

asociaremos dos conjuntos fundamentales:

$$W_i = \{x_i\}, \quad R_i = \text{Vars}(e_i),$$

donde $\text{Vars}(e_i)$ denota el conjunto de variables que aparecen en el lado derecho e_i .

Intuitivamente, W_i registra la información que la instrucción produce, mientras que R_i registra la información que consume. Bajo las hipótesis del modelo, estos conjuntos bastan para detectar interferencias semánticamente relevantes entre pares de instrucciones.

6 Dependencias de datos y de salida

Sean s_i y s_j dos instrucciones tales que $i < j$. Diremos que el orden relativo entre ellas es obligatorio si al intercambiarlas podría cambiar el estado final del programa. Hay tres casos canónicos.

6.1 Dependencia RAW

La dependencia *read-after-write* aparece cuando la segunda instrucción lee una variable escrita por la primera.

$$W_i \cap R_j \neq \emptyset.$$

Como $W_i = \{x_i\}$, esto equivale a exigir $x_i \in R_j$.

Ejemplo 6.1 (Dependencia RAW). Considérese

$$s_1 : a := b + c, \quad s_2 : d := a + 1.$$

La instrucción s_1 escribe a y s_2 lee a . Si se invirtiera el orden, s_2 observaría el valor viejo de a en lugar del recién producido por s_1 . Luego el orden s_1 antes que s_2 es obligatorio.

6.2 Dependencia WAR

La dependencia *write-after-read* aparece cuando la primera instrucción lee una variable que la segunda sobrescribe.

$$R_i \cap W_j \neq \emptyset.$$

Como $W_j = \{x_j\}$, esto equivale a $x_j \in R_i$.

Ejemplo 6.2 (Dependencia WAR). Considérese

$$s_1 : a := b + 1, \quad s_2 : b := 0.$$

La instrucción s_1 necesita leer el valor original de b , mientras que s_2 lo sobrescribe. Si se intercambiaran, s_1 leería el nuevo valor 0. Por tanto, el orden relativo también es obligatorio.

6.3 Dependencia WAW

La dependencia *write-after-write* aparece cuando ambas instrucciones escriben la misma variable.

$$W_i \cap W_j \neq \emptyset.$$

En nuestro caso, esto equivale simplemente a $x_i = x_j$.

Ejemplo 6.3 (Dependencia WAW). Considérese

$$s_1 : a := 1, \quad s_2 : a := 2.$$

En el orden original, el valor final de a es 2; si se invierte el orden, el valor final pasa a ser 1. El orden relativo debe preservarse.

7 Restricciones de precedencia

Definición 7.1 (Restricción de precedencia). Para $i < j$, escribiremos

$$s_i \rightarrow s_j$$

si entre s_i y s_j existe una dependencia RAW, WAR o WAW. Equivalentemente, imponemos la precedencia $i \rightarrow j$ si

$$W_i \cap R_j \neq \emptyset \quad o \quad R_i \cap W_j \neq \emptyset \quad o \quad W_i \cap W_j \neq \emptyset.$$

Estas restricciones capturan los pares de instrucciones cuyo orden relativo no puede modificarse sin riesgo semántico.

8 El grafo de precedencia

Definición 8.1 (Grafo de precedencia). El grafo de precedencia asociado a la secuencia $S = (s_1, \dots, s_n)$ es el grafo dirigido

$$G_S = (V, E)$$

donde

$$V = \{1, \dots, n\}$$

y $(i, j) \in E$ si y solo si $i < j$ y existe la restricción de precedencia $s_i \rightarrow s_j$.

Cada vértice representa una instrucción. Cada arista indica que el orden entre esos dos vértices debe preservarse en cualquier reordenamiento correcto.

Lema 8.2. *El grafo de precedencia G_S es acíclico.*

Demostración. Por construcción, toda arista $(i, j) \in E$ satisface $i < j$. En consecuencia, a lo largo de cualquier camino dirigido los índices crecen estrictamente. Si existiera un ciclo

$$i_1 \rightarrow i_2 \rightarrow \cdots \rightarrow i_k \rightarrow i_1,$$

obtendríamos

$$i_1 < i_2 < \cdots < i_k < i_1,$$

lo cual es imposible. Luego G_S es un DAG. □

Observación 8.3. El grafo no impone un orden total, sino parcial. Entre dos nodos no conectados puede existir libertad de reordenamiento. La idea central del método es que toda la información semánticamente necesaria queda resumida en este orden parcial.

9 Conmutación local de instrucciones independientes

Antes de enunciar el resultado principal, conviene estudiar el caso de dos instrucciones adyacentes.

Proposición 9.1 (Criterio local de conmutación). *Sean*

$$s \equiv x := e \quad y \quad t \equiv y := f.$$

Las secuencias $s; t$ y $t; s$ son semánticamente equivalentes si se cumplen las tres condiciones siguientes:

- (I) $x \neq y$,
- (II) $x \notin \text{Vars}(f)$,
- (III) $y \notin \text{Vars}(e)$.

Demostración. La condición (ii) asegura que la evaluación de f no depende del valor escrito por s , mientras que la condición (iii) asegura que la evaluación de e no depende del valor escrito por t . La condición (i) evita que una escritura sobrescriba a la otra. En consecuencia, ejecutar s y luego t , o bien t y luego s , asigna exactamente los mismos valores finales a x y a y , y deja intactas las demás variables. Luego ambas secuencias son semánticamente equivalentes. □

10 Ordenamientos topológicos

Definición 10.1 (Ordenamiento topológico). *Sea $G = (V, E)$ un DAG. Un ordenamiento topológico de G es una lista lineal*

$$v_1, \dots, v_n$$

que contiene todos los vértices de V y satisface que, para toda arista $(u, v) \in E$, el vértice u aparece antes que v en la lista.

En nuestro contexto, un ordenamiento topológico es precisamente un reordenamiento de las instrucciones que respeta todas las restricciones de precedencia.

11 Teorema principal

Teorema 11.1 (Caracterización exacta). *Sea $S = (s_1, \dots, s_n)$ una secuencia de asignaciones puras y sea G_S su grafo de precedencia. Una permutación de S preserva la semántica si y solo si el orden inducido sobre los índices $\{1, \dots, n\}$ es un ordenamiento topológico de G_S .*

Demostración. La prueba se divide en dos direcciones.

(Solo si). Supóngase que una permutación preserva la semántica pero viola alguna arista $i \rightarrow j$ del grafo. Entonces en la permutación resultante la instrucción s_j aparece antes que s_i , a pesar de que entre ellas existe una dependencia RAW, WAR o WAW.

- Si la dependencia es RAW, s_j lee una variable que debía ser producida por s_i , luego la lectura observa el valor equivocado.
- Si la dependencia es WAR, s_j sobrescribe una variable que s_i debía leer antes, luego la lectura observa el valor equivocado.
- Si la dependencia es WAW, el valor final de la variable escrita cambia porque se altera cuál escritura ocurre de último.

En cualquiera de los tres casos, existe un estado inicial para el cual el estado final cambia. Contradicción.

(Si). Supóngase ahora que una permutación corresponde a un ordenamiento topológico de G_S . Mostraremos que puede obtenerse a partir del orden original mediante una secuencia finita de intercambios de pares adyacentes que no tienen dependencia entre sí.

Cada vez que dos instrucciones adyacentes aparecen en el orden inverso respecto del orden objetivo, pero sin violar ninguna arista del DAG, necesariamente no están conectadas por una restricción de precedencia. Por el criterio local de conmutación, ese intercambio preserva la semántica. Iterando el procedimiento, se transforma el orden original en el orden topológico deseado sin alterar la semántica en ningún paso. Por composición, la permutación final también preserva la semántica. $\square$

Corolario 11.2. *Las permutaciones semánticamente válidas son exactamente las extensiones lineales del orden parcial inducido por el grafo de precedencia.*

12 Algoritmos

12.1 Construcción del grafo

La construcción del grafo de precedencia es cuadrática en el número de instrucciones: basta comparar cada par (i, j) con $i < j$.

```

Entrada:  $s_1, \dots, s_n$  con  $s_i = x_i := e_i$ 
Salida:  $G = (V, E)$ 

 $V := \{1, \dots, n\}$ 
 $E := \text{conjunto\_vacío}$ 

Para cada  $i$ :
     $W_i := \{x_i\}$ 
     $R_i := \text{Vars}(e_i)$ 

Para cada par  $i, j$  con  $1 \leq i < j \leq n$ :
    si  $(W_i \cap R_j \text{ no es vacío})$  o  $(R_i \cap W_j \text{ no es vacío})$ 
        o  $(W_i \cap W_j \text{ no es vacío})$ :
             $E := E \cup \{(i, j)\}$ 

retornar  $(V, E)$ 

```

12.2 Enumeración de todas las permutaciones válidas

Como G_S es un DAG, podemos enumerar todos sus ordenamientos topológicos mediante backtracking, eligiendo en cada paso cualquier vértice de indegree cero todavía no emitido.

```

EnumerarTopSorts( $G$ , orden_actual):
    si todos los vertices ya fueron emitidos:
        emitir orden_actual
        retornar

     $Z := \text{conjunto de vertices no emitidos con indegree } 0$ 

    para cada  $v$  en  $Z$ :
        marcar  $v$  como emitido
        eliminar temporalmente sus aristas salientes
        EnumerarTopSorts( $G_{\text{reducido}}$ , orden_actual ++ [ $v$ ])
        restaurar aristas y desmarcar  $v$ 

```

Observación 12.1. El número de ordenamientos topológicos puede crecer exponencialmente. Esto no es un defecto del algoritmo, sino una propiedad intrínseca del problema: cuando hay mucha independencia real, existen muchas permutaciones semánticamente válidas.

13 Ejemplos completos

13.1 Ejemplo I: una dependencia RAW y una WAR

Considérese la secuencia

$$s_1 : a := b + c,$$

$$s_2 : d := a + 1,$$

$$s_3 : b := 0.$$

Se tiene

$$W_1 = \{a\}, R_1 = \{b, c\}, \quad W_2 = \{d\}, R_2 = \{a\}, \quad W_3 = \{b\}, R_3 = \emptyset.$$

Las dependencias son:

- $1 \rightarrow 2$ por RAW, ya que $a \in R_2$.
- $1 \rightarrow 3$ por WAR, ya que $b \in R_1$ y s_3 escribe b .
- No hay arista entre 2 y 3.

Luego el grafo tiene las aristas

$$1 \rightarrow 2, \quad 1 \rightarrow 3.$$

Sus ordenamientos topológicos son exactamente

$$(1, 2, 3) \quad \text{y} \quad (1, 3, 2).$$

Estas son, por tanto, las únicas permutaciones semánticamente válidas.

13.2 Ejemplo II: dos instrucciones independientes

Considérese la secuencia

$$s_1 : x := a + b,$$

$$s_2 : y := c + d,$$

$$s_3 : z := x + y.$$

Las aristas del grafo son

$$1 \rightarrow 3, \quad 2 \rightarrow 3,$$

y no existe arista entre 1 y 2. En consecuencia, los ordenamientos topológicos son

$$(1, 2, 3) \quad \text{y} \quad (2, 1, 3).$$

Este ejemplo ilustra que la independencia semántica se traduce en libertad de intercalación.

13.3 Ejemplo III: una cadena forzada por WAR

Considérese

$$s_1 : x := y + 1,$$

$$s_2 : y := z + 1,$$

$$s_3 : z := 0.$$

Aquí se obtiene

$$1 \rightarrow 2 \quad \text{y} \quad 2 \rightarrow 3,$$

ambas por WAR. El único ordenamiento topológico es entonces $(1, 2, 3)$, de modo que no existe libertad de reordenamiento.

14 Discusión y alcance

El enfoque presentado es exacto bajo las hipótesis del modelo porque, en ese marco, la interacción entre instrucciones queda completamente determinada por los conjuntos de lectura y escritura. Sin embargo, fuera de este contexto conviene introducir refinamientos.

- **Efectos laterales.** Si las expresiones realizan E/S, modifican memoria o pueden fallar de manera observable, las dependencias puramente sintácticas ya no bastan.
- **Aliasing.** Si dos nombres pueden denotar la misma celda mutable, una escritura puede afectar lecturas aparentemente no relacionadas.
- **Semánticas observacionales más finas.** Si además del estado final importa el número de pasos, el momento de una excepción o el orden exacto de ciertos eventos, la noción de equivalencia debe ajustarse.

También es útil distinguir entre equivalencia semántica y análisis puramente sintáctico. Por ejemplo, una instrucción como $x := x$ no altera el estado, pero un análisis basado solo en conjuntos de lectura y escritura la tratará como una escritura genuina. Incorporar este tipo de simplificaciones requiere un razonamiento semántico más refinado sobre las expresiones.

15 Conclusión

El problema de generar todas las permutaciones semánticamente válidas de una secuencia de asignaciones puras admite una solución limpia, completa y algorítmicamente útil. La receta es la siguiente:

1. para cada instrucción, calcular qué variables lee y qué variable escribe;
2. para cada par de instrucciones, introducir una arista de precedencia cuando exista dependencia RAW, WAR o WAW;
3. observar que el grafo resultante es un DAG;
4. enumerar todos sus ordenamientos topológicos.

El resultado final es una caracterización exacta: las permutaciones válidas son exactamente las extensiones lineales del orden parcial inducido por las dependencias. Esta correspondencia enlaza de manera natural la semántica operacional de secuencias de asignaciones con técnicas combinatorias clásicas sobre DAGs.

A Resumen ejecutivo del método

Dada una secuencia de asignaciones

$$s_i \equiv x_i := e_i,$$

defínase

$$W_i = \{x_i\}, \quad R_i = \text{Vars}(e_i).$$

Para cada par $i < j$, agréguese la arista $i \rightarrow j$ si ocurre alguna de las siguientes condiciones:

$$x_i \in R_j, \quad x_j \in R_i, \quad x_i = x_j.$$

El grafo obtenido es acíclico y las secuencias válidas son exactamente sus ordenamientos topológicos.